Pierwsza aplikacja w Ionic React

Na tych zajęciach tworzymy pierwszy projekt **Ionic** z biblioteką **React** i budujemy w nim prosty kalkulator BMI. Poznamy:

- tworzenie i uruchamianie projektu narzędziem Ionic CLI,
- podstawowe komponenty Ionic: nagłówek, siatkę (`IonGrid`), pola tekstowe, przyciski, ikony i karty,
- odczytywanie wartości z pól (`useRef`) i wyświetlanie wyniku (`useState`),
- podgląd aplikacji tak, jak wygląda na telefonie.

Kod oddajemy w repozytorium w folderze **`lab_04/`** (patrz „Oddanie" na końcu).

Przygotowanie

W pracowni Node.js i Ionic CLI są już zainstalowane — sprawdzamy tylko wersje:

```
node --version
ionic --version
```

Praca w domu: jeśli `node` nie działa, instalujemy aktualną wersję LTS ze strony https://nodejs.org/en/download/ (po instalacji trzeba otworzyć terminal na nowo). Ionic CLI instalujemy poleceniem:

```
npm install -g @ionic/cli
```

Szczegółowe informacje w dokumentacji: https://ionicframework.com/docs/intro/cli

Tworzenie projektu

Projekt tworzymy w folderze **poza folderami synchronizowanymi przez OneDrive, Dropbox itp.** (na Windowsie często dotyczy to Pulpitu i Dokumentów) — np. w `C:\dev`. Program synchronizujący blokuje pliki w `node_modules` i narzędzia zgłaszają błędy (`EBUSY: resource busy or locked` albo `EPERM: operation not permitted`).

Uwaga: `ionic start` sam tworzy folder `lab_04`. Polecenie wpisujemy więc w **głównym folderze repozytorium** z laboratoriami (tam, gdzie są `lab_01`, `lab_02` …), a nie w samodzielnie utworzonym folderze `lab_04` — inaczej powstanie `lab_04\lab_04`. Otwieramy terminal w głównym folderze repozytorium i wpisujemy:

```
ionic start lab_04 blank --type=react --no-git
```

Polecenie tworzy folder `lab_04` z projektem React (`--type=react`) na pustym szablonie (`blank`). Opcja `--no-git` wyłącza tworzenie osobnego repozytorium Git wewnątrz projektu — kod i tak trafi do naszego repozytorium z laboratoriami. CLI od razu dodaje do projektu **Capacitor** (pakiety `@capacitor/…` i plik `capacitor.config.ts`) — to narzędzie do budowania aplikacji na Androida i iOS, przyda się na kolejnych zajęciach. Na koniec CLI zapyta `Create free Ionic account?` — odpowiadamy `n` (konto nie jest potrzebne).

Instalacja zależności może zająć kilka minut. Ostrzeżenia `npm warn deprecated`, informację o `vulnerabilities` i `npm notice New … version of npm available` możemy zignorować. Z listy „next steps" wypisanej na końcu przez CLI korzystamy tylko z dwóch pierwszych (`cd`, `ionic serve`) — pozostałych (`ionic capacitor add`, `cordova-res`) dziś nie wykonujemy.

Jeśli `ionic start` przerwie się błędem (np. `EBUSY` / `EPERM`), usuwamy utworzony folder `lab_04` i wpisujemy polecenie jeszcze raz — w folderze poza OneDrive/Dropboxem. Jeśli powstał folder `lab_04\lab_04`, najprościej usunąć go w całości i powtórzyć `ionic start` we właściwym miejscu.

Samo `ionic start` (bez argumentów) zadaje kolejno pytania: `Use the app creation wizard?` (kreator na stronie internetowej, wymaga konta Ionic — odpowiadamy `n`), `Framework:` (wybieramy **React**), `Project name:` (`lab_04`) i `Starter template:` (**blank**).

Uruchomienie aplikacji

Przechodzimy do folderu projektu i uruchamiamy serwer deweloperski:

```
cd lab_04
ionic serve
```

Aplikacja otworzy się w przeglądarce pod adresem http://localhost:8100. Pusty projekt pokazuje stronę z nagłówkiem **Blank** i — na środku ekranu — napisem „Ready to create an app?" z linkiem do listy komponentów Ionic:

https://ionicframework.com/docs/components

Kolory aplikacji (jasne lub ciemne) zależą od ustawień systemu — szablon włącza automatyczny tryb ciemny, więc przy ciemnym motywie Windowsa tło będzie prawie czarne. To nie błąd.

W konsoli przeglądarki (F12 → **Console**) mogą się pojawić ostrzeżenia **React Router Future Flag Warning** — dotyczą routingu, który za chwilę usuniemy, i znikną po porządkowaniu projektu.

Porządkowanie projektu

W pliku `src/App.tsx` zostawiamy importy plików CSS i wywołanie `setupIonicReact()`, a usuwamy routing i stronę `Home`. Najprościej zamienić całą zawartość pliku na:

```
import { IonApp, setupIonicReact } from '@ionic/react';

/* Core CSS required for Ionic components to work properly */
```

```tsx
import '@ionic/react/css/core.css';

/* Basic CSS for apps built with Ionic */
import '@ionic/react/css/normalize.css';
import '@ionic/react/css/structure.css';
import '@ionic/react/css/typography.css';

/* Optional CSS utils that can be commented out */
import '@ionic/react/css/padding.css';
import '@ionic/react/css/float-elements.css';
import '@ionic/react/css/text-alignment.css';
import '@ionic/react/css/text-transformation.css';
import '@ionic/react/css/flex-utils.css';
import '@ionic/react/css/display.css';

import '@ionic/react/css/palettes/dark.system.css';

/* Theme variables */
import './theme/variables.css';

setupIonicReact();

const App: React.FC = () => (
  <IonApp>
    <h2>abc</h2>
  </IonApp>
);

export default App;
```

Dopiero po zapisaniu `App.tsx` usuwamy foldery `src/pages` i `src/components` (wcześniej `App.tsx` jeszcze z nich korzysta). Pliku `src/main.tsx` nie zmieniamy — uruchamia on komponent `App`. Pliki testów (`App.test.tsx`, `setupTests.ts`, folder `cypress`) zostawiamy, nie będziemy z nich korzystać.

Strona w przeglądarce odświeży się sama i powinien pojawić się na niej napis `abc`. Od teraz każda zmiana zapisana w kodzie jest od razu widoczna w przeglądarce — nie trzeba ponownie uruchamiać `ionic serve`. Jeśli strona zrobi się biała, w kodzie jest błąd — szukamy go w konsoli przeglądarki (F12 → **Console**) i w terminalu.

Podgląd jak na telefonie

Ionic jest przeznaczony głównie do aplikacji mobilnych, dlatego aplikację oglądamy w widoku telefonu: w Chrome **najpierw** otwieramy DevTools (F12), a potem klikamy w panelu DevTools ikonę **Toggle device toolbar** (telefon z tabletem, w lewym górnym rogu panelu) albo naciskamy Ctrl+Shift+M — skrót działa tylko przy otwartych DevTools (bez nich Chrome otwiera menu profilu). Następnie wybieramy z listy u góry dowolny telefon z grupy **Mobile** (np. Pixel albo iPhone — lista modeli zmienia się z wersjami Chrome).

Komponenty Ionic zmieniają wygląd zależnie od platformy: na Androidzie i w przeglądarce mają styl **Material Design** (`md`), na iPhonie — styl **iOS** (`ios`). Styl iOS możemy podejrzeć bez iPhone'a: wpisujemy w pasku adresu http://localhost:8100/?ionic:mode=ios i zatwierdzamy Enterem (strona musi się

przeładować). Na samym napisie `abc` różnica jest niewielka (inna czcionka); wyraźnie widać ją po wykonaniu zadania 1:

Material Design (`md`)	iOS (`ios`)
tytuł w nagłówku z lewej strony	tytuł wyśrodkowany
napisy na przyciskach WIELKIMI LITERAMI	napisy zwykłymi literami
przyciski prostokątne, z cieniem	przyciski mocniej zaokrąglone, płaskie

Aby wrócić do stylu `md`, usuwamy z adresu `?ionic:mode=ios` i ponownie zatwierdzamy Enterem.

Zad 1 kalkulator

Utwórz prosty kalkulator BMI.

Importy. Każdy użyty komponent Ionic trzeba dodać do importu z `@ionic/react` w **pierwszej linii** pliku `App.tsx`. Nie dopisujemy drugiej linii `import … from '@ionic/react'` (dwa importy tej samej nazwy dają błąd) — dopisujemy nowe nazwy po przecinku w nawiasie klamrowym istniejącej linii, np.:

```
import { IonApp, IonContent, IonHeader, setupIonicReact } from '@ionic/react';
```

Edytor podpowiada import, gdy zaczniemy wpisywać nazwę komponentu — sprawdźmy wtedy, czy dopisał nazwę do istniejącej linii.

Budujemy stronę krok po kroku:

1. Zamiast `<h2>abc</h2>` dodaj dwa elementy: `IonHeader` oraz `IonContent`.
2. Wewnątrz `IonHeader` dodaj `IonToolbar`, a wewnątrz niego `IonTitle` z tytułem, np. `Kalkulator BMI`.
3. Do elementu `IonContent` dodaj klasę `ion-padding` (`className="ion-padding"`).
4. Wewnątrz `IonContent` dodaj siatkę `IonGrid`, a w niej wiersz `IonRow`. Wewnątrz wiersza dodaj kolumnę `IonCol`, a wewnątrz kolumny — `IonItem`. W elemencie `IonItem` dodaj pole `IonInput`:

   ```
   <IonInput label="Wzrost (cm)" labelPlacement="floating" type="number" />
   ```

   Właściwość `label` to etykieta pola, a `labelPlacement="floating"` sprawia, że po kliknięciu etykieta „odpływa" nad wpisywany tekst.
5. **Drugie pole:** dodaj taką samą strukturę (kolejny wiersz `IonRow` → `IonCol` → `IonItem` → `IonInput`) dla wagi, z etykietą `Waga (kg)`.

Na stronie powinny być teraz nagłówek z tytułem i **dwa** pola: wzrost i waga.

Poniżej, nadal wewnątrz `IonGrid`, dodaj kolejny wiersz na przyciski. A wewnątrz dodaj kolumnę z przyciskiem do obliczania:

```
<IonCol>
  <IonButton>
    <IonIcon slot="start" />
    Oblicz
  </IonButton>
</IonCol>
```

W tym samym wierszu dodaj kolumnę na przycisk do resetowania wyników:

```
<IonCol>
  <IonButton>
    <IonIcon slot="start" />
    Resetuj
  </IonButton>
</IonCol>
```

Do naszych kolumn możemy dodać klasy: `ion-text-left` (pierwsza) oraz `ion-text-right` (druga), aby przyciski znalazły się przy przeciwnych krawędziach ekranu.

Następnie zaimportuj ikony (to osobny import — z pakietu `ionicons/icons`, nie z `@ionic/react`):

```
import { calculatorOutline, refreshOutline } from 'ionicons/icons';
```

Do obu elementów `IonIcon` dodaj ikonę za pomocą właściwości `icon`: przy „Oblicz" `icon={calculatorOutline}`, przy „Resetuj" `icon={refreshOutline}`. Właściwość `slot="start"` umieszcza ikonę przed tekstem przycisku.

Listę dostępnych ikon znajdziemy na stronie https://ionic.io/ionicons. **Uwaga:** strona pokazuje użycie w czystym HTML-u (`<ion-icon name="archive-outline">`). W Ionic React właściwość `name` nie działa (ikona się nie pojawi, a w konsoli zobaczymy ostrzeżenie `Setting the "name" property has no effect`). Nazwę ze strony zamieniamy na zapis camelCase, importujemy ją z `ionicons/icons` i podajemy w `icon`: `archive-outline` → `import { archiveOutline } from 'ionicons/icons';` i `icon={archiveOutline}`.

Aby przyciski coś robiły, potrzebujemy funkcji. Zmieniamy zapis komponentu `App` tak, aby przed zwracanym kodem JSX można było umieścić instrukcje — zamiast nawiasu okrągłego `=> (` używamy klamry i słowa `return`:

```
const App: React.FC = () => {
  const oblicz = () => {};
  const resetuj = () => {};

  return (
    <IonApp>
      {/* dotychczasowa zawartość */}
```

```
    </IonApp>
  );
};
```

Do przycisku z obliczaniem dodaj zdarzenie `onClick`, które wywołuje funkcję `oblicz`, a do przycisku z resetowaniem — funkcję `resetuj`. Dopisujemy je wewnątrz istniejącego znacznika `<IonButton>`, np. przycisk „Oblicz" wygląda teraz tak:

```
<IonButton onClick={oblicz}>
  <IonIcon slot="start" icon={calculatorOutline} />
  Oblicz
</IonButton>
```

Uwaga: przekazujemy samą nazwę funkcji, bez nawiasów. Zapis `onClick={oblicz()}` wywołałby funkcję od razu przy rysowaniu strony, a nie po kliknięciu.

Aby można było pobrać wartość z pola, musimy skorzystać z hooka `useRef`. Na początku pliku dodajemy:

```
import { useRef } from 'react';
```

Na początku komponentu `App` (przed funkcjami) dodaj dwie stałe — referencje do pól wzrostu i wagi:

```
const wzrostRef = useRef<HTMLIonInputElement>(null);
const wagaRef = useRef<HTMLIonInputElement>(null);
```

Zapis `<HTMLIonInputElement>` informuje TypeScript, że referencja będzie wskazywać na element `IonInput`, a `null` to wartość początkowa (zanim element pojawi się na stronie). Referencje dodaj do pól za pomocą właściwości `ref`, np. `<IonInput ... ref={wzrostRef} />`.

W funkcji `oblicz` dodaj następujący kod:

```
const oblicz = () => {
  const wzrost = wzrostRef.current!.value;
  const waga = wagaRef.current!.value;
  if (!wzrost || !waga) {
    return;
  }
  const wzrostM = +wzrost / 100;
  const bmi = +waga / (wzrostM * wzrostM);
  console.log(bmi);
};
```

Wykrzyknik informuje TypeScript, że wartość `current` nie będzie w tym miejscu równa `null`. Znak `+` przed zmienną zamienia jej wartość (tekst) na liczbę. Wzrost wpisujemy w centymetrach, a wzór na BMI

wymaga metrów — stąd dzielenie przez 100. Wynik sprawdzamy w konsoli przeglądarki (F12 → **Console**), np. dla 180 cm i 75 kg BMI wynosi ok. 23,15.

Do funkcji `resetuj` dodaj kod, który przypisze polom (`wzrostRef.current!.value`, `wagaRef.current!.value`) wartości równe `''`.

Zad 2

Dodaj wynik poniżej przycisków. Aby to zrobić, musimy skorzystać z hooka stanu `useState`.

Najpierw import: linię `import { useRef } from 'react';` zamieniamy na:

```
import { useRef, useState } from 'react';
```

(bez tego zobaczymy białą stronę, a w konsoli `ReferenceError: useState is not defined`). Do importu z `@ionic/react` dopisujemy też `IonCard` i `IonCardContent`.

Do siatki, poniżej wiersza z przyciskami, dodaj kolejny wiersz:

```
<IonRow>
  <IonCol>
    <IonCard>
      <IonCardContent>
        <h2>BMI</h2>
      </IonCardContent>
    </IonCard>
  </IonCol>
</IonRow>
```

Wewnątrz elementu `h2` umieścimy nasze BMI.

Na samym początku komponentu `App` dodajmy stan dla naszego BMI — zmienną i funkcję, która ją zmienia (tzw. setter):

```
const [obliczoneBMI, setObliczoneBMI] = useState<number>();
```

Na końcu funkcji `oblicz` dodajemy linijkę, która zapisze obliczone BMI w stanie:

```
setObliczoneBMI(bmi);
```

Zmiana stanu powoduje ponowne narysowanie komponentu. Nagłówek w karcie zmieniamy tak, aby wypisywał wartość BMI:

```
<h2>BMI: {obliczoneBMI.toFixed(2)}</h2>
```

Metoda `toFixed(2)` zaokrągla wynik do dwóch miejsc po przecinku.

Wynik ma się wyświetlać tylko wtedy, gdy BMI zostało obliczone. Korzystamy z konstrukcji `{warunek && (...)}`: jeżeli lewa strona jest prawdziwa (tu: `obliczoneBMI` ma wartość), wyświetlone zostanie to, co znajduje się po prawej stronie; w przeciwnym razie nic się nie pojawi. Cały wiersz z kartą obejmujemy więc zapisem:

```
{obliczoneBMI && (
  <IonRow>
    {/* ... */}
  </IonRow>
)}
```

Na koniec w funkcji `resetuj` dopisz linijkę, dzięki której przycisk „Resetuj" ukryje również wynik:

```
setObliczoneBMI(undefined);
```

Sprawdzamy: przed pierwszym obliczeniem karty nie ma, po kliknięciu **Oblicz** pojawia się wynik (np. `BMI: 23.15` dla 180 cm i 75 kg), a **Resetuj** czyści pola i ukrywa kartę.

Zad 3 kalkulator jako osobny komponent

Utwórz prosty kalkulator na wzór kalkulatora z Windowsa. Tym razem nie piszemy go w `App.tsx`, tylko jako **osobny komponent** — dzięki temu `App.tsx` pozostanie czytelny, a na kolejnych zajęciach przeniesiemy kalkulator do innej zakładki aplikacji.

Utwórz folder `src/components`, a w nim plik `Kalkulator.tsx`. Komponent budujemy tak samo jak `App`, a na końcu pliku go eksportujemy:

```
import { IonCard, IonCardContent } from '@ionic/react';

const Kalkulator: React.FC = () => {
  return (
    <IonCard>
      <IonCardContent>
        {/* wyświetlacz i przyciski */}
      </IonCardContent>
    </IonCard>
  );
};

export default Kalkulator;
```

W `App.tsx` importujemy go (`import Kalkulator from './components/Kalkulator';`) i wstawiamy w `IonContent`, poniżej siatki kalkulatora BMI, jak każdy inny komponent: `<Kalkulator />`.

Wymagania:

- **Wyświetlacz** — tekst (np. w `<h1>`) pokazujący wpisywaną liczbę albo wynik. Jego wartość trzymamy w stanie (`useState`), a nie w polu `IonInput` — w zadaniu 1 odczytywaliśmy wartość z pola przez `useRef`, tutaj to stan decyduje o tym, co widać na ekranie.
- **Przyciski** ułożone w siatce `IonGrid` w czterech kolumnach, jak w kalkulatorze z Windowsa:

  ```
  7  8  9  ÷
  4  5  6  ×
  1  2  3  −
  C  0  .  +
        =
  ```

- **Działanie**: wpisujemy liczbę, wybieramy działanie, wpisujemy drugą liczbę, klikamy `=` i widzimy wynik. Wynik można wykorzystać w kolejnym działaniu (np. `9 − 4 =`, potem `× 3 =` daje 15). `C` czyści wszystko. Druga kropka w jednej liczbie jest ignorowana.
- **Dzielenie przez zero** — zamiast wyniku wyświetlacz pokazuje napis „Nie można dzielić przez zero".
- **Nie używamy** `eval()` (ani `new Function()`) — funkcja wykonująca dowolny tekst jako kod JavaScript jest niebezpieczna, a działanie liczymy sami, np. instrukcją `if` albo `switch`.

Wskazówka: wystarczą trzy stany — tekst na wyświetlaczu, poprzednia liczba (`useState<number | null>(null)`) i wybrane działanie (`useState<string | null>(null)`). Kliknięcie działania zapamiętuje liczbę z wyświetlacza i działanie, a `=` liczy wynik z poprzedniej liczby, działania i liczby z wyświetlacza. Wyniki takie jak `0.1 + 0.2 = 0.30000000000000004` możemy zaokrąglić: `parseFloat(wynik.toFixed(10))`.

Dla chętnych: przyciski `±`, `%` i `⌫` (usuwanie ostatniej cyfry), liczenie łańcuchowe jak w Windowsie (`2 + 3 +` od razu pokazuje 5), obsługa klawiatury.

Oddanie

Kod umieszczamy w repozytorium GitHub w folderze `lab_04/` — wszystko, co jest potrzebne do uruchomienia aplikacji:

- **oddajemy:** foldery `src/` i `public/`, pliki `index.html`, `package.json`, `package-lock.json`, pozostałe pliki konfiguracyjne z głównego folderu projektu (`*.json`, `*.ts`, `*.js`, `.gitignore`, `.browserslistrc`) oraz `README.md`,
- **nie oddajemy:** folderów `node_modules/` (setki MB — odtwarza je `npm install`) i `dist/` (wynik budowania) ani testów z szablonu: folderu `cypress/` i pliku `cypress.config.ts`.

`node_modules` i `dist` są wpisane w `.gitignore`, więc `git add` je pomija. Testy dopisujemy do `.gitignore` sami — na końcu pliku dodajemy dwie linie:

```
/cypress
/cypress.config.ts
```

Uwaga: przy wgrywaniu plików przez stronę GitHuba (**Add file** → **Upload files**) `.gitignore` **nie działa** — wtedy wymienionych folderów i plików po prostu nie przeciągamy.

W pliku `lab_04/README.md` opisujemy, jak uruchomić aplikację (`npm install`, `ionic serve`), i dołączamy zrzuty ekranu: kalkulator BMI z wynikiem (w widoku telefonu) oraz kalkulator z zadania 3 z wynikiem działania.

Źródła:

- https://ionicframework.com/docs/intro/cli
- https://ionicframework.com/docs/components
- https://ionicframework.com/docs/api/input
- https://ionicframework.com/docs/api/grid
- https://ionic.io/ionicons
- https://react.dev/reference/react/useRef
- https://react.dev/reference/react/useState